# DS1307 Real-Time Clock, I2C Subsystem & Power-Switch Architecture

## 1. Overview of the DS1307 RTC

The **DS1307** is a low-power, full binary-coded decimal (BCD) clock/calendar real-time clock IC manufactured by Analog Devices (formerly Maxim Integrated). It counts seconds, minutes, hours, day of the week, date of the month, month, and year with leap-year compensation valid up to the year 2100.

---

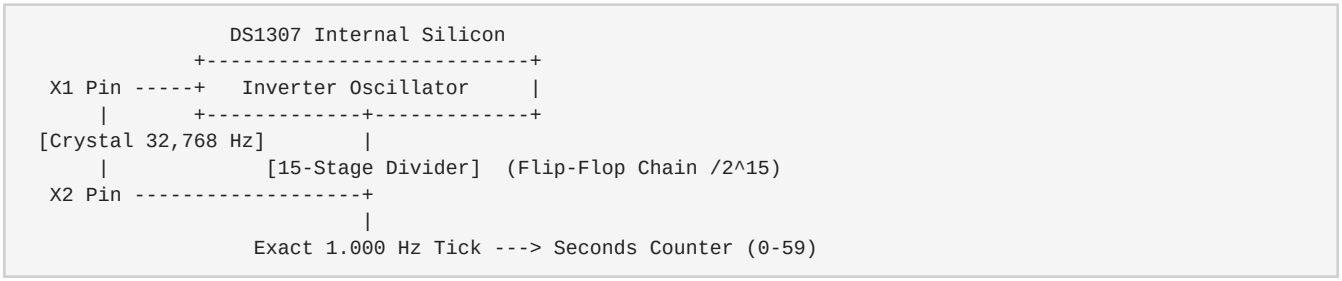
## 2. Technical Specifications & Operating Limits

| Parameter | Datasheet Specification | Engineering Details |
|---|---|---|
| **Interface Protocol** | I2C Serial Bus (Standard Mode) | 100 kHz bus clock frequency |
| **I2C Device Address** | 7-bit: **`0x68`** | 8-bit shifted: **`0xD0`** (Write), **`0xD1`** (Read) |
| **Operating Voltage ($V_{CC}$)** | **4.5 V to 5.5 V** (Nominal: 5.0 V) | **Strict requirement**: Fails at 3.3V |
| **Backup Battery ($V_{BAT}$)** | 2.0 V to 3.5 V | CR2032 3.0V Lithium coin cell |
| **Backup Battery Current** | **$< 500\,\text{nA}$** (Typ: $300\,\text{nA}$) | Enables **10+ years** of continuous timekeeping |
| **Oscillator Frequency** | 32.768 kHz | Parallel resonant quartz crystal ($C_L = 12.5\,\text{pF}$) |
| **Internal RAM** | 56 bytes battery-backed SRAM | Registers **`0x08`** to **`0x3F`** |

---

## 3. Quartz Crystal Oscillation Physics: Why 32.768 kHz?

The time base of the DS1307 is derived from a $32.768\,\text{kHz}$ quartz crystal connected across pins **`X1`** and **`X2`**:




### Mathematical Principle:

$$32,768 = 2^{15}$$

By feeding this frequency into a **15-stage binary ripple counter** (a cascade of 15 toggle flip-flops), the frequency is divided by 2 at each stage:

$$f_{\text{output}} = \frac{32,768\,\text{Hz}}{2^{15}} = 1.000\,\text{Hz}\quad (\text{Exact } 1\text{-second period})$$

• **Why not a 1 MHz or 8 MHz crystal?**: Dynamic CMOS power consumption is governed by $P = C \cdot V^2 \cdot f$. Operating at $32.768\,\text{kHz}$ reduces switching losses to nanowatts, allowing a tiny CR2032 coin cell to sustain the clock for over a decade.

---

## 4. Internal Register Map & Binary-Coded Decimal (BCD)

The timekeeper registers occupy addresses **`0x00`** through **`0x07`**:

| Address | Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 | Function | Range | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **`0x00`** | **CH** | 10 Seconds | Seconds | Seconds | Seconds | Seconds | Seconds | Seconds | Seconds & Clock Halt | 00–59 | |
| **`0x01`** | 0 | 10 Minutes | Minutes | Minutes | Minutes | Minutes | Minutes | Minutes | Minutes | 00–59 | |
| **`0x02`** | 0 | **12/24** | 10 Hr/PM | 10 Hr | Hours | Hours | Hours | Hours | Hours | 00–23 (or 1–12) | |
| **`0x03`** | 0 | 0 | 0 | 0 | 0 | Day | Day | Day | Day of Week | 1–7 | |
| **`0x04`** | 0 | 0 | 10 Date | Date | Date | Date | Date | Date | Date of Month | 01–31 | |
| **`0x05`** | 0 | 0 | 0 | 10 Month | Month | Month | Month | Month | Month | 01–12 | |
| **`0x06`** | 10 Year | Year | Year | Year | Year | Year | Year | Year | Year | Year | 00–99 |
| **`0x07`** | OUT | 0 | 0 | SQWE | 0 | 0 | RS1 | RS0 | Control Register | - | |

### 4.1 BCD Arithmetic & Bit-Shift Macros

In BCD encoding, each decimal digit is stored in an independent 4-bit nibble:

• Decimal **`45`**: High nibble = **`4`** (**`0100b`**), Low nibble = **`5`** (**`0101b`**) $\rightarrow$ Stored as **`0x45`**.

• Our driver provides sub-nanosecond hardware bit-shift converters:

```
#define BCD_TO_DEC(val) (((val) >> 4) * 10 + ((val) & 0x0F))
#define DEC_TO_BCD(val) ((((val) / 10) << 4) | ((val) % 10))
```

### 4.2 The Clock Halt (**`CH`**) Flag

• **Bit 7 of Register `0x00`** is the **Clock Halt (CH)** bit.

• When a fresh coin battery is inserted at the factory, the oscillator is halted by default (**`CH = 1`**) to preserve battery charge during shipping.

• In [**`DS1307_Init()`**](file:///c:/Users/GOPIKRISHNA/Downloads/MC_project_Pill_Box/MC_project_Pill_Box/Core/Src/ds1307.c), the firmware reads register **`0x00`**, clears bit 7 (**`buf[0] &= 0x7F`**), and writes it back, guaranteeing the quartz oscillator starts ticking immediately.

---

## 5. Power-Fail Comparator & Battery-Switch Circuit

The DS1307 contains internal power-sensing circuitry that monitors $V_{CC}$ relative to $V_{BAT}$:

```
     VCC (5.0V from USB)
           |
       [Comparator] <--- Compares VCC against 1.25 * VBAT (~3.75V)
           |
     +-----+-----+
     |           |
If VCC > 1.25*VBAT:                    If VCC < 1.25*VBAT (Power Cut):
- Powered from VCC                     - Switched to VBAT (CR2032)
- I2C Bus ACTIVE                       - I2C Bus INHIBITED (Write Protected)
- Full Read/Write Access                - Ultra-Low Power Mode (<500 nA)
```

### Critical Engineering Lesson for CSE Students:

- If $V_{CC}$ is connected to the Blue Pill's **3.3V rail**:

$$V_{CC} = 3.3\text{V} < 1.25 \times 3.0\text{V} = 3.75\text{V}$$

The internal comparator falsely trips into **battery-backup mode**, permanently locking the I2C bus and returning **`RTC: NONE`**!

- **Correct Wiring**: $V_{CC}$ must be tied to the **5V pin** (fed from USB), while **PB6** and **PB7** communicate safely via open-drain 3.3V pull-ups.

---

## 6. The "Tiny RTC" Breakout Board Diode Hazard

Standard hobbyist "Tiny RTC" modules feature an unbuffered charging circuit (diode **D1** and resistor **R5**) intended for rechargeable **LIR2032 (3.6V)** lithium-ion cells.

- If a common disposable **CR2032 (3.0V primary lithium)** cell is used, the 5V rail continuously forces reverse charging current into the non-rechargeable cell, causing battery bulging, electrolyte leakage, or rupture.
- **Engineering Fix**: Desolder diode **D1** or remove resistor **R5** from the module to completely isolate the primary CR2032 cell.

---

## 7. Evaluator Oral Questions & Detailed Answers

### Q1: Why can the 3.3V STM32 communicate safely with the 5V DS1307 over I2C?

> ***Answer**:*
>
> *I2C pins **PB6** (SCL) and **PB7** (SDA) on the STM32F103 are **Five-Volt Tolerant (FT)** pins configured in **Alternate Function Open-Drain (`AF_OD`)** mode. Neither device drives voltage onto the lines. The bus voltage is determined entirely by the pull-up resistors. Because our pull-up resistors are tied to the **3.3V rail**, the bus voltage swings between $0\text{V}$ and $3.3\text{V}$. The DS1307 datasheet specifies an input high threshold of $V_{IH,\min} = 2.2\text{V}$. Because $3.3\text{V} > 2.2\text{V}$, the DS1307 recognizes every HIGH logic level with full noise margin and zero risk of overvoltage.*

### Q2: Why does the firmware use burst I2C reading instead of separate register reads?

> ***Answer**:*

*If registers are read individually in 7 separate transactions, a rollover could occur between reading seconds (e.g. at 07:59:59) and reading minutes, resulting in an erroneous timestamp like 07:00:59. In* **`DS1307_GetTime()`**, *we send register address* **`0x00`** *and read all 7 bytes in a single burst:*

```c
HAL_I2C_Mem_Read(hi2c, DS1307_I2C_ADDR, 0x00, 1, buf, 7, 100);
```

*The DS1307 automatically latches its internal registers into a shadow buffer during I2C access, preventing timekeeper roll-over corruption.*

### Q3: How does the system handle an RTC hardware failure or disconnected battery?

***Answer**:*

*The firmware incorporates a dual-mode timekeeper architecture. If* **`DS1307_GetTime()`** *fails to acknowledge over I2C,* **`rtcAvailable`** *is set to* **`false`**, *an alert icon (*`!`*) appears on the OLED header, and the super-loop automatically invokes software tick interpolation using* **`HAL_GetTick()`**, *ensuring scheduled alarms continue to trigger without freezing the device.*